

Quant Signals — The Complete Function Reference

Every function in the system, explained so you could rebuild it from this document alone. Companion to `internals.md` (the conceptual tour — read it first for why; this file is the what and how, module by module, function by function). Code lives in `quant_signals/` (the library), `scripts/` (things cron runs), and `app.py` (the dashboard).

The primer — five things everything below assumes you know:

1. A **DataFrame** (pandas) is a table: rows have an index (here, usually dates), columns have names (usually tickers or feature names). A **Series** is a single column. "Wide" means dates × tickers.
2. A **MultIndex panel** is a Series/DataFrame indexed by two things at once — here `(date, ticker)`. One row = one stock on one day. `.xs(d, level="date")` slices out one day; `.groupby(level="date")` computes something per day.
3. **Parquet** is a fast binary file format for tables. `.to_parquet()` / `pd.read_parquet()` — that's it.
4. A **z-score** is `(value - mean) / standard deviation` — "how many standard deviations from average." Z-scoring across stocks on one day turns any feature into a relative rank-like number.
5. **Basis points (bps)**: 1 bp = 0.01%. Costs here are 10 bps = 0.10%.

Conventions: every function that takes prices expects the **wide closes frame** (dates × tickers, split/dividend-adjusted). Scores and features are **(date, ticker) panels**. Everything time-sensitive obeys one law: a number stamped with date `t` may only be computed from data up to `t`'s close.

1. `quant_signals/config.py` — every knob in one place

No functions, only constants. Everything else imports these; change them here and the whole system follows.

Constant	Value	Meaning
<code>PROJECT_ROOT</code>	(derived)	The repo directory, found relative to this file.
<code>DATA_DIR</code>	<code>data_cache/</code>	Where downloaded prices live.
<code>OUTPUT_DIR</code>	<code>outputs/</code>	

Constant	Value	Meaning
		Where each week's scores/metrics land (overwritten weekly).
HISTORY_YEARS	10	Depth of the first-ever price download.
LABEL_HORIZON	10	Trading days ahead the model predicts (~2 weeks).
PURGE_DAYS	= LABEL_HORIZON	Gap between training data and scored dates (leak guard, §6).
TRAIN_WINDOW_DAYS	756	~3 years of training history per monthly refit.
MIN_TRAIN_DAYS	252	Skip a refit with less than ~1 year of history.
TOP_FRACTION	0.10	Buy the top decile of the ranking.
EXIT_FRACTION	0.20	...but hold a name until it falls out of the top 20% (buffer band).
REGIME_SMA	200	SPY moving-average window for the crash filter.
REGIME_SCALE	0.5	Exposure when SPY is below that average.
COST_BPS	10.0	Trading cost charged per unit of traded weight.
REBALANCE_FREQ	"W-FRI"	Rebalance on Fridays.
BENCHMARK	"SPY"	The index everything is compared to.
CHAMPION	"ml"	The one strategy treated as "the" model. Changes only with explicit user approval.
DIAGNOSTICS_DIR	diagnostics/	Weekly point-in-time archive (§12).
M1_DATE	"2026-10-16"	The report in §13 refuses the real ledger before this date.
M1_BENCHMARKS	("SPY", "RSP", "MTUM")	Pre-named benchmark battery. Do not extend.

2. `quant_signals/data/universe.py` — who's in the game

`tickers_from_table(df) -> list[str]` Input: a DataFrame with a `Symbol` column (a table scraped from Wikipedia). Output: sorted, deduplicated ticker list in Yahoo

format. It drops NaN rows (one Wikipedia table has an annotation row) and rewrites `.` as `-` (`BRK.B` → `BRK-B`) because Yahoo spells share classes with a dash.

`_fetch_tables(url) -> list[DataFrame]` Downloads a Wikipedia page and returns every HTML table on it as a DataFrame (`pd.read_html`). The one trick: it sends a browser-like `User-Agent` header, because Wikipedia answers the default Python one with HTTP 403. (The leading underscore = "private helper, not part of the module's public face" — a convention used throughout.)

`_sp500_table() -> DataFrame` Finds the S&P 500 constituents table by its `Symbol` column (not by page position) and sanity-checks 450–550 unique symbols, raising `RuntimeError` otherwise — the same fail-loudly guard the Russell-1000 fetch always had. Production runs unattended every Friday; a reshuffled Wikipedia page must abort the run, not silently shrink the universe.

`get_sp500_tickers() -> list[str]` The universe: current S&P 500 members from `_sp500_table`, passed through `tickers_from_table`. ~503 tickers. Caveat baked into the whole system: this is today's list — survivorship bias; see internals §2.1.

`get_sp500_sectors() -> dict[str, str]` Same guarded table, but returns `{ticker: GICS_sector}` (GICS = the standard industry classification, e.g. "Information Technology"). Used by the M1 report to measure sector concentration. Saved to `data_cache/sectors.json`.

`get_russell1000_tickers() -> list[str]` Same idea for the Russell 1000 (used only by research round 5). The components table isn't first on that page, so it searches for the table having a `Symbol` column, and sanity-checks that it got 900–1100 names — if Wikipedia reshuffles the page, fail loudly rather than train on garbage.

3. `quant_signals/data/prices.py` — the price cache

`with_retries(fn, attempts=3, base_delay=2.0)` Calls `fn()`; on exception waits 2 s, 4 s, 8 s (exponential backoff) and retries; re-raises the last error. Wraps network downloads because Yahoo hiccups.

`merge_frames(old, new) -> DataFrame` Cell-level merge of two wide frames: `new.combine_first(old)` — a new value wins where new has one; a NaN or missing column in `new` **never erases** a real value in `old`. The comment in the code calls this scar tissue: an earlier row-level version once NaN-wiped cached history. The cache is precious; this function is why it can't be destroyed by one bad download.

`download_prices(tickers, start) -> (closes, volumes)` One yfinance call for all tickers since `start`, `auto_adjust=True` (splits and dividends baked in). Returns the wide closes and volumes frames, drops all-NaN rows, and logs a warning listing tickers that returned nothing.

`_basis_break_tickers(old, new, rtol=BASIS_RTOL) -> list[str]` The adjusted-basis tripwire. yfinance's `auto_adjust` rebases a ticker's **entire** history at every split/dividend, so the narrow incremental window is only safe to overlay while the fresh

data still agrees with the cache on their overlapping dates. This compares each known ticker cell-wise over the overlap and returns those disagreeing beyond `BASIS_RTOL` (1 bp — dividends move the basis by $\geq \sim 10$ bps, float noise by far less). Without it, a 2:1 split would freeze a permanent fake -50% day into the cache at the refetch boundary, poisoning momentum/vol features and training labels.

`_download_incremental(tickers, old_closes, old_volumes) -> (closes, volumes, downloaded_anything)` The cache-refresh brain. Splits the request in two: - **known** tickers (already cached): re-download only the last ~ 7 days — cheap, and re-fetching a week catches late dividend adjustments; - **new** tickers (just added to the index): download the full 10 years, so their long-lookback features (252-day momentum) aren't NaN for months.

Known data is merged with `merge_frames` (can't clobber); new columns are outer-joined. Tickers flagged by `_basis_break_tickers` are **excluded from the window overlay** (never freeze a mixed basis) and instead re-downloaded over the cache's full date span; their cached closes and volumes columns are replaced wholesale. If the repair download returns nothing for a ticker, its cached history is left untouched — stale but on one consistent basis — and re-detected on the next run. If `known` is empty it skips that download entirely — calling yfinance with `[]` crashes. The third return value says whether the network returned anything at all (window, new, or repair), so the caller can distinguish "Yahoo is down" from "nothing to fetch."

`update_price_cache(tickers, cache_dir=DATA_DIR) -> None` The public entry. No cache yet $\rightarrow$ full 10-year download; empty result raises `RuntimeError` rather than writing a poisoned empty cache. Cache exists $\rightarrow$ `_download_incremental`; if nothing came back, log a warning and **leave the existing files untouched**. On success writes `closes.parquet`, `volumes.parquet`, and `tickers.json` (the universe as of this download).

`load_cached(cache_dir=DATA_DIR) -> (closes, volumes, tickers)` Reads those three files back. That's the whole data API the rest of the system uses.

4. `quant_signals/features/indicators.py` — price $\rightarrow$ signal ingredients

All six operate on the whole wide frame at once (every stock, every day — no loops over tickers) and are pure functions: same input, same output. `TRADING_DAYS = 252` (a year of market days).

`momentum(closes, window)` — trailing return over `window` days: `closes / closes.shift(window) - 1`. `.mask(base <= 0)` blanks entries where the old price was a bad print (zero/negative). Used with windows 21/63/126/252 — and with window 5, where it's actually a reversal signal (recent sharp risers tend to dip back; the baseline weights it negatively).

`rsi(closes, window=14)` — Relative Strength Index, 0–100: how one-sided recent daily moves were. Split each day's change into gains and losses, average each over the window (Cutler's variant: simple moving average), then `100 - 100/(1 + gain/`

`loss`). All-gain windows would divide by zero; they're forced to exactly 100 ("pure uptrend").

`realized_vol(closes, window)` — annualized volatility: standard deviation of daily % changes over the window $\times \sqrt{252}$. (Variance grows with time, so std grows with its square root — the standard annualization.)

`sma_distance(closes, window)` — % distance from the simple moving average: `closes / rolling_mean - 1`. Positive = above trend.

`high_proximity(closes, window=252)` — % below the rolling 52-week high: `closes / rolling_max - 1`. 0 = at the high (a classic bullish anomaly), -0.4 = 40% below it.

`volume_zscore(volumes, window=63)` — is today's volume unusual for this stock: `(volume - rolling mean) / rolling std`.

5. `quant_signals/features/panel.py` — assembling model food

FEATURES — the canonical list of 12 column names: `mom_21, mom_63, mom_126, mom_252, rev_5, rsi_14, vol_21, vol_63, sma_50, sma_200, high_252, volz_63`. (`rev_5` is `momentum(closes, 5)` — the name records how it's used, not how it's computed.)

`build_feature_panel(closes, volumes) -> DataFrame` Computes all 12 indicators, stacks the wide frames into one long panel indexed `(date, ticker)` with one column per feature, names the index levels, and **drops any row with a missing value** (a stock too young for 252-day momentum simply isn't in the panel yet). Every feature at date *t* uses data through *t* only — the indicators are all backward-looking by construction.

`zscore_by_date(panel) -> DataFrame` For each date, for each feature: subtract that day's cross-stock mean, divide by that day's cross-stock std, clip to ± 5 . After this a value means "vs. the other stocks today", never an absolute level — the mechanical enforcement of "everything is relative" (internals §3.2).

`forward_demeaned_returns(closes, horizon=10) -> Series` The **label** — the answer key the model trains on. For each `(date t, ticker)`: the return from `close(t+1)` to `close(t+1+horizon)`, minus the average of that same return across all stocks that day. `shift(-1)` is the no-time-travel wall (you trade the close after the signal, so no price at or before *t* is in the label); demeaning strips the market move so the label is pure relative performance, matching the z-scored features. Returns a `(date, ticker)` Series named `"label"`. The most dangerous 8 lines in the repo — internals §3.3 walks them slowly.

6. `quant_signals/model/` — the three scorers

A "scorer" maps the z-scored panel to a **score** per `(date, ticker)`; higher = expected to do relatively better over the next ~2 weeks.

`baseline.py`

`BASELINE_WEIGHTS` — seven fixed weights on z-scored features, signs straight from published anomaly research: momentum 63/126/252 and 52-week high positive, 5-day reversal and 63-day volatility negative, 200-day trend positive. Nothing is fitted.

`baseline_scores(zpanel) -> Series` — literally `sum(weight × zpanel[feature])`. One line. Its whole job is to be the hurdle: if the ML model can't beat this, the ML is worthless.

`ml.py`

`walk_forward_splits(dates, train_window_days=756, purge_days=10, min_train_days=252)` A generator yielding `(train_dates, score_dates)` for each calendar month. For a month starting at index `first` in the sorted unique dates: train slice = `dates[start:end]` with `end = first - purge_days - 1` and `start = end - train_window_days`. So the newest training date sits `purge_days + 2` positions before the first scored date — and since each training label peeks `purge_days + 1` days forward, the newest label ends **one day before** the scored month begins. No overlap = no leak. Months with under `min_train_days` of history are skipped (yields nothing).

`_make_model(params=None)` — builds a fresh `HistGradientBoostingRegressor` (sklearn's gradient-boosted trees: many small decision trees, each correcting the last — a strong non-linear curve-fitter). Defaults: `max_depth=3, max_iter=200, learning_rate=0.05, l2_regularization=1.0, random_state=0` — deliberately shallow/regularized, because the signal-to-noise here is tiny and a flexible model will happily memorize noise. `params` lets research rounds override; `random_state=0` makes every run reproducible.

`walk_forward_scores(zpanel, labels, ...) -> (scores, final_model, final_train_frame)` The training loop. Joins panel+labels (inner join = only rows that have both), then for each split: fit a fresh model on the training window, predict scores for the month's dates, collect. Note the asymmetry: models train only on labeled rows, but score every panel row in the month — including the newest dates whose labels don't exist yet. Those unlabeled predictions **are the live signal** you trade. Returns the concatenated scores plus the last fitted model and its training frame (kept for feature importance). Feature columns are taken from the panel, so research panels with extra columns work unchanged. If there's not enough history for even one split: warn and return an empty Series rather than crash.

`trees.py`

The champion is retrained every run and **never pickled**, so without this module there is no record of what the forest actually looked like. It turns the fitted estimator into a JSON-able payload at train time (`outputs/model_trees.json`) that the dashboard reads back.

`HistGradientBoostingRegressor` is awkward to introspect: unlike the classic `GradientBoostingRegressor` it exposes no `estimators_`, and neither `plot_tree` nor `export_text` accepts it. Its trees live as a private array of node records (`model._predictors`), one flat record per node with fields `value / count / feature_idx / num_threshold / left / right / gain / is_leaf`.

`_predictor_nodes(model) -> list` — the **only** place that touches that private API, deliberately funnelled through one function. Raises `AttributeError` if `_predictors` is gone or a node field was renamed; `test_trees.py` pins the record shape so an sklearn upgrade breaks the test suite rather than a Friday-night run.

`_walk(nodes, i, features) -> dict` — recursion turning the flat node array into a nested dict from node `i`: leaves become `{leaf, n}`, internal nodes `{feature, thr, gain, n, missing_left, left, right}`.

`extract_trees(model, features, trained_through="", train_rows=0) -> dict` Returns `{"summary": ..., "trees": [...]}`. Alongside the nested trees the summary carries the estimator's own parameters, `baseline_prediction`, and three per-feature tallies computed in one pass: `split_counts / split_share` (how often a feature is split on), `gain_share` (its share of total split gain), and `root_counts` (how many trees it roots). Two reading rules: thresholds are in the panel's units — production z-scores per date, so a threshold is **standard deviations from that date's cross-section**, not a raw return — and leaf values are **already shrunk by the learning rate**, so a prediction is `baseline_prediction + one leaf per tree`.

`render_tree_html(payload, theme="light") -> str` — inlines the payload into `tree_view.html` (a self-contained page: no CDN, no external fonts) and pins `theme` to the host dashboard's palette; the page also honours `prefers-color-scheme` when opened standalone. Refuses to render a payload that could break out of its `<script>` tag.

`tree_view.html` — the viewer itself. Node-link diagram of one tree at a time over the whole forest ($\leftarrow \rightarrow$, slider, jump box, hover tooltips), a per-tree readout (root split, leaf count, strongest leaf), and a split-share chart contrasting gain share against split-count share. Structural only: it says what the model splits on, never whether the splits pay.

`blend.py`

`blend_scores(ml, baseline) -> Series` Converts each scorer's output to within-day percentile ranks (top stock = 1.0, median = 0.5) — making two incomparable scales comparable — then averages them. Covers only rows present in both (it inherits ML's later start). A diversification bet: two mediocre-but-different signals averaged are often steadier than either.

7. `quant_signals/backtest/engine.py` — the honest simulator

`_target_names(day_scores, held, top_fraction, exit_fraction) -> Index` Buffer-band selection, the single most important structural idea (worth more Sharpe than any modeling change — research round 2): 1. Rank today's scores, best = rank 1 (`method="first"` breaks ties deterministically). 2. Target book size = `top_fraction` × universe (min 1). 3. **Keep** every currently-held name still ranked inside the wider `exit_fraction` band. 4. **Fill** remaining seats with the best-ranked unheld names.

A stock must fall out of the top 20% to be sold, not merely out of the top 10% — names hovering at the boundary stop flip-flopping, roughly halving turnover. Both the backtest and the live paper ledger import this exact function, so simulation and reality provably share selection logic.

`regime_exposure(benchmark_closes, window=200, scale=0.5) -> Series` The crash seatbelt. 1.0 when SPY closes at/above its 200-day moving average, else 0.5 (hold half size). Backward-looking; during the SMA warm-up the comparison is False so exposure stays 1.0 (NaN-free).

`run_backtest(scores, closes, top_fraction=0.10, cost_bps=10, rebalance_freq="W-FRI", exit_fraction=0.20, exposure=None) -> (daily_returns, turnover)` Steps through every trading day with four pieces of state: `active` (today's actual weights), `pending` (a rebalance decided but not yet executed), `held` (current names), and the running `daily` returns. Each day, in order: 1. Earn today's return: `sum(active_weights × today's % changes)`. 2. If `pending` exists (decided yesterday): execute it **now** — `turnover = Σ|new - old| weight changes`, `cost = turnover × cost_bps / 104`, `active = pending`. 3. If today is a rebalance date (Friday): pick names via `_target_names`, scale equal weights by the regime filter, park as `pending` — to execute tomorrow. 4. Record `ret - cost`.

Decide Friday, trade at the next close — the same one-day wall as the label (\$5), so the backtest measures exactly what the label taught. Weights sit static between rebalances (drift ignored — a simplification). A NaN price contributes 0% that day ("frozen cash": fine for current-constituent data, would hide collapses if delisted names were ever added — documented in the docstring). Raises `ValueError` if `exposure` contains NaN, because a NaN scale would silently corrupt the books.

8. `quant_signals/backtest/metrics.py` — the scoreboard

`cagr(returns)` — compound the daily series, then take the `1/years`-th root – 1: the smooth annual rate that would produce the same final wealth.

`sharpe(returns)` — `mean / std × √252`: return per unit of risk, annualized; the headline number (SPY ≈ 0.8; 1.0 is genuinely good). Zero-variance series return 0.0 instead of dividing by zero.

`max_drawdown(returns)` — build the wealth curve, track its running peak (`cummax`), take the deepest % fall below it. The "could you stomach it?" number.

`summary(returns) -> dict` — all three, keyed `cagr` / `sharpe` / `max_drawdown`.

9. `quant_signals/paper.py` — the un-fudgeable ledger

Positions written **before** outcomes exist, graded later, append-only, in git.

`LEDGER_DIR = ledger/`; two CSVs with fixed schemas: `positions.csv` = `score_date, strategy, ticker, weight, exposure`; `performance.csv` = `score_date, strategy, entry_date, exit_date, gross_return, cost, net_return` (doesn't exist until the first settlement).

`_load_csv(path, cols, date_cols)` — reads a ledger CSV; missing file → empty frame with the right columns; **wrong columns** → `ValueError` ("ledger corrupt or from an incompatible version") — the schema check runs before date parsing so pandas' own errors can't mask it.

`_write_ledger_csv(df, path, cols, date_cols)` The only way ledger CSVs reach disk. Writes to a temp file, re-reads it, verifies every previously recorded row survives the rewrite unchanged (raising `ValueError` and leaving the original byte-identical otherwise), then swaps atomically with `os.replace` — append-only enforced at the writer, and no crash mid-write can truncate committed history.

`load_positions(ledger_dir=LEDGER_DIR)` / `load_performance(ledger_dir=LEDGER_DIR)`

The two public readers (the M1 fixtures pass a different dir; production uses the default).

`_latest_book(positions, strategy, before=None)` — a strategy's most recently recorded book, optionally strictly before a date (that option is what settlement uses to find the prior book for turnover). Empty frame if none.

`build_books(scores_by_strategy, exposure_scale, prior_positions) -> DataFrame` For each strategy: take its newest score date, slice that day's scores, seed `held` from the strategy's last recorded book, run the **engine's** `_target_names` (imported as the single source of truth), and emit equal-weight rows scaled by `exposure_scale`. Raises if a scorer has no scores on its date — better no book than a wrong book.

`record(scores_by_strategy, exposure_scale, ledger_dir=LEDGER_DIR) -> bool` Appends this week's books to `positions.csv` (via `_write_ledger_csv`). Refuses to write a "mixed-date" book if the scorers disagree on the newest date (a partial retrain would cause this). **Idempotent:** if this `score_date` is already recorded, return False and touch nothing — that's what makes the weekly cron safe to re-run after a crash.

`_entry_date(closes_index, score_date)` — the first trading day strictly after a date; None if it doesn't exist yet. Encodes the same next-close entry rule as the backtest and the label.

`_price_asof(closes, ticker, date)` — last available close at or before `date`; NaN if none. Tolerates a holding that didn't trade on the exact day.

`_book_return(book, closes, entry, exit_)` — $\sum \text{weight} \times (\text{exit price} / \text{entry price} - 1)$ over the book's rows; a holding with a missing price logs a warning and contributes 0% (same "frozen cash" convention as the engine).

`_turnover(book, prior)` — $\sum |\text{weight changes}|$ across the union of old and new holdings (an exited name counts its full old weight, an entered name its full new weight) — matching the engine's cost model exactly.

`settle_pending(closes, ledger_dir=LEDGER_DIR) -> int` The grading step. A book recorded at `score_date` `t` enters at the first close after `t` and exits at the first close after the **next** book's score date — held exactly one rebalance period, like the backtest. For every recorded book that isn't settled yet and whose exit close now exists: gross return via `_book_return`, cost via `_turnover` $\times 10$ bps, append `net_return = gross - cost` to `performance.csv` (via `_write_ledger_csv`). Also writes one **SPY row** per settled week (costless — the "just buy the index" comparison). Idempotent via the set of already-settled `(score_date, strategy)` keys. The newest book never settles (no successor yet) — which is why the ledger always trails by ~2 weeks.

10. `quant_signals/pipeline.py` — the conductor

CLI: `python -m quant_signals.pipeline <command>`. Each `cmd_*` is one stage; `main()` is an argparse dispatch table mapping command names to them. Friday's cron calls them in this order (via `weekly.sh`):

`cmd_update_data()` — S&P 500 tickers $\cup \{\text{SPY}\} \rightarrow \text{update_price_cache}.$

`_universe_frames()` — load the cache, drop SPY and anything without a closes column; returns the closes/volumes the model may see (SPY is the benchmark, never a holding).

`cmd_train()` — the full modeling pass: build panel $\rightarrow$ z-score $\rightarrow$ labels $\rightarrow$ `baseline_scores` $\rightarrow$ `walk_forward_scores` (minutes) $\rightarrow$ `blend_scores`; writes `outputs/scores_{baseline,ml,blend}.parquet`. Then persists the newest date's feature cross-section to `outputs/features_latest.parquet` (for the diagnostics snapshot, §12) and computes **permutation feature importance** on a $\leq 20\text{k}$ -row sample of the final training set $\rightarrow$ shuffle one feature's column, see how much prediction degrades; repeat 3 $\times$; average — written to `outputs/feature_importance.json`. Finally `extract_trees` (§6) dumps the final model's forest to `outputs/model_trees.json` for the dashboard. That last step is wrapped in `try/except` and logged: it is a diagnostic nothing downstream reads, so a renderer or sklearn-layout failure must never cost the run its scores.

`_regime_exposure()` — SPY closes from cache $\rightarrow$ `regime_exposure`.

`cmd_backtest()` — run all three strategies through `run_backtest` with the regime filter; raise if any score file is empty. Every metric is computed **on the ML strategy's date window** (ML starts ~1 year late — it needs its first training

window; comparing unmatched windows would be apples-to-oranges). Adds a SPY daily-return column, average weekly turnover per strategy; writes `outputs/backtest_returns.parquet` (the four equity curves) and `outputs/metrics.json`.

`cmd_robustness()` — three stress tests, same matched window: **double_costs** (20 bps — does the edge survive?), **by_year** (metrics per calendar year — is one lucky year carrying it?), **veterans_only** (only stocks with ≥ 2400 days of history — a partial probe of survivorship bias). Writes `outputs/robustness.json`.

`cmd_mark()` — load closes, `settle_pending` — grade whatever became gradable this week.

`cmd_record()` — load the three score files, read the regime filter's scale at the score date (recorded with the book — the ledger stores the actual exposure taken), `paper.record`.

`cmd_snapshot()` — same exposure lookup, then `diagnostics.snapshot_week` (\$12). Runs right after `record`; failure is non-fatal in `weekly.sh`.

11. `scripts/` — what cron actually runs

`weekly.sh` (Fridays 23:00 UTC) — `set -euo pipefail`, cd to repo, then: a **preflight** (refuses to run if `quant_signals/`, `scripts/`, or `pyproject.toml` differ from HEAD — uncommitted code must not steer the recorded book), the six pipeline stages in the order above (snapshot's failure swallowed with `|| echo WARNING...`), the two non-fatal monitors (`price_spotcheck.py` after update-data, `sector_alert.py` after snapshot), then four commit blocks, each conditional on something having changed: (1)

`ledger/` — commit+push, (2) the news store parquet, (3) `diagnostics/`, (4) append a run summary (per-strategy Sharpe, recorded/settled counts — computed by an inline Python heredoc) to `~/brain/capture/quant-signals-runs.md` and push the vault, (5) re-render and push the public status page. Every step is idempotent, so a partial failure heals on the next run. Log: `logs/weekly.log`. **Failure alert (added 2026-07-10):** a `STEP` variable is set before every stage and an EXIT trap fires when the script dies with `STEP != finished`, pushing a Telegram message that names the failed step (via `notify_owner.py`, best-effort — the alert can never break the run).

`sector_alert.py` (weekly, non-fatal) — `top_sector(positions, sectors)`: (date, sector, share) of the latest champion book's largest sector by summed weight; `main()` pages when share > 50% (the M2 memo's stop-tightening threshold). Monitoring only until R11 is decided at M1; reads the ledger, changes nothing. `--dry-run` prints.

`price_spotcheck.py` (weekly, non-fatal) — `fetch_alpaca_daily(tickers)`: ~2 weeks of daily closes per ticker in one batched request to Alpaca's **data** endpoint (IEX feed; yfinance-style `BRK-B` translated to Alpaca's `BRK.B` and back; no trading surface touched). `main()` samples 8 random cached names, compares the latest shared close, pages on > 2% divergence. Independent-vendor check the basis guard can't do; alert-only so a data hiccup never creates a missed Friday. Note: stooq was the

first choice and blocks this server — the discovery that led to the M1 benchmark fallback being re-pointed at this same Alpaca data API (2026-07-15, see `fetch_benchmark_closes`).

`notify_owner.py` — `notify(text)` : best-effort Telegram push to the owner, shared by the ops scripts; reuses the brain-dispatch bot env (`~/.config/brain-dispatch/env` , 0600, never git) exactly like `execute_paper.notify` . Never raises; returns False and prints a warning on failure. Also a CLI: `notify_owner.py "message"` .

`deadman_check.py` (Saturdays 06:00 UTC) — `checks(today)` : alert strings when the ledger's newest `score_date` is not in the just-passed Friday's **ISO week** (date equality would false-alarm on holiday Thursdays like 2026-07-02). Catches what the weekly.sh trap cannot: the run never starting (crontab lost, box rebooted wrong). `main()` sends via `notify` , or prints with `--dry-run` . Read-only — it defends N (a silently missed Friday becomes a stretched row excluded from M1 inference), and changes nothing (amendment-closure compatible). Residual gap, accepted: it runs on the box it watches, so a dead box alerts nobody (off-site mirror proposed and declined 2026-07-10; the Saturday Mac sync is the off-box copy).

`collect_news.py` (daily 21:45 UTC) — the pure data asset (not wired to the model; a future research round's raw material). - `fetch_rows(tickers)` — for each ticker, `yf.Ticker(t).news` ; each item → one row: `ticker, id, pub_ts, title, summary, provider, fetch_ts` . `fetch_ts` (now, UTC) is the point-in-time guarantee: whatever happens later, we can prove this headline was captured before it. 0.25 s sleep between tickers (politeness); failures counted, not fatal. - `main()` — read the universe from `tickers.json` , fetch, concat with the existing store, dedupe on `(ticker, id)` keeping the **first** fetch (preserving the earliest `fetch_ts`), then **atomic write**: write to a temp file, `os.replace` over the store — an interrupted run can never half-write the irreplaceable history. Empty fetch → exit without touching the store.

`collect_estimates.py` (daily 09:00 UTC) — round-15's pure data asset (adopted 2026-07-15; not wired to the model — first read at M-earn $\geq$ 2027-07-15 per the R15 spec in `~/quant-research`). - `fetch_rows(tickers)` — one pass over the universe via `yf.Ticker(t).earnings_dates` : names reporting within the next 7 calendar days → snapshot rows

(`ticker, report_ts, report_date, eps_estimate, fetch_ts, fetch_date`) building a T-7...T-1 consensus drift series per event; reports in the trailing 10 days whose `Reported EPS` has appeared → actual rows (`... reported_eps, eps_estimate_at_fetch, surprise_pct ...`). `fetch_ts` is the PIT guarantee, exactly like the news store. 0.25 s sleep; failures counted, not fatal. - `_append(store, rows, keys)` — append-only merge, first-write-wins dedupe (snapshots on `(ticker, report_date, fetch_date)` , actuals on `(ticker, report_date)`), atomic temp-then- `os.replace` write. - `main()` — universe from `tickers.json` ; both fetches empty → exit without touching the stores (Yahoo-down safety: skip, never fabricate). Stores live in `data_cache/earnings_estimates/` , git-tracked (`.gitignore` carve-out) and committed weekly by `weekly.sh` (`estimates-commit` step) — the news store's 3-place durability scheme.

`render_status.py` — builds the public status page (aggregates only, never individual positions). - `scoreboard_rows(metrics)` — metrics.json → HTML table rows; champion highlighted, SPY dimmed. - `curve_svg(returns_path)` — the backtest equity curves as a hand-built SVG polyline chart (downsampled 5:1 — a full ~8k-point daily curve once crashed mobile browsers), champion drawn last/thickest, dashed line at 1.0x. - `ledger_block()` — recorded/settled week counts from the ledger. - `ledger_fingerprint()` (added 2026-07-10) — SHA-256 hex of each ledger CSV (`positions.csv` , `performance.csv` ; missing files skipped — the performance line appears at the first settlement). Published on the page so GitHub's commit history third-party-timestamps the record weekly: any retro-edit changes the fingerprint. A hash reveals nothing about positions, so the aggregates-only rule holds. - `research_block()` — round-4 sweep progress if the research repo has results. - `main()` — pour all of it into `status_template.html` placeholders, write `index.html` (the weekly run pushes it to `~/status-site`).

12. `quant_signals/diagnostics.py` — the black-box recorder

`snapshot_week(output_dir=OUTPUT_DIR, diag_dir=DIAGNOSTICS_DIR, exposure_scale=1.0)`
-> `Path | None` `outputs/` is overwritten every Friday; this preserves each week's **pre-outcome** artifacts before that happens, into `diagnostics/<score_date>/` (git-tracked): - `scores.parquet` — all three strategies' newest-date scores, long format `strategy, ticker, score` ; - `features.parquet` — copy of `features_latest.parquet` (skipped with a warning if absent); - `feature_importance.json` — copied; - `meta.json` — score date, the regime exposure actually applied, ticker count.

Refuses a **mixed-date** snapshot if the scorers' newest dates disagree (same guard as `record`). Idempotent: destination dir exists → return `None`. Everything stored was computed before outcomes existed — there is nothing here to peek at; that's what makes an honest October drift analysis possible (realized IC, importance stability) instead of relying on overwritten files or memory.

13. `quant_signals/ml_report.py` — the pre-registered October read

Blind-built (developed on synthetic fixtures only), date-gated, verdicts pre-committed. CLI: `python -m quant_signals.ml_report [--fixtures DIR] [--output PATH]` .

Statistics core

`bootstrap_ci(diffs, n_boot=10_000, alpha=0.10, seed=0)` — 90% percentile confidence interval on the mean of weekly (champion – SPY) differences: resample the N observed values with replacement 10,000 times, take each resample's mean, return the 5th and 95th percentiles of those means. No bell-curve assumption; fixed seed = reproducible. If the interval contains 0, the record can't rule out "no edge."

`min_detectable_edge(diffs)` — $(1.645 + 0.842) \times \text{std}/\sqrt{N}$: the smallest true weekly edge a one-sided 5% test would detect with 80% power at the observed N. (The bare $1.645 \times \text{std}/\sqrt{N}$ is only the rejection threshold — an edge exactly there is detected half the time; relabelled 2026-07-08 at 0 settled weeks.) Printed so the reader sees the test's power — at $N \approx 15$ this is much larger than any plausible edge, which is exactly why V-ALPHA forbids decisions.

`bootstrap_sharpe_ci(weekly, ...)` — same bootstrap, but the statistic is the annualized ($\sqrt{52}$) Sharpe of the champion's weekly nets; zero-std resamples score 0.0.

`factor_verdict(champ_weekly, bench_weekly)` — the pre-committed labeling rule, CI-gated since 2026-07-08 (0 settled weeks): returns `(label, stats)` where `stats[etf] = (mean, lo, hi)` is the 90% bootstrap CI on each champion-minus-ETF weekly diff. SPY diff CI entirely below zero → **BEHIND**; entirely above zero → **ALPHA-CONSISTENT** if every ETF's diff CI excludes zero upward, else **FACTOR-CARRIED**; SPY CI straddling zero → **INDETERMINATE** (no label asserted from bare point means).

`champion_beta(table, champ)` — descriptive OLS beta of champion weekly nets on SPY. Joint-aligns the pair first (a benchmark-skipped week leaves a NaN SPY row that crashed the old separate-`dropna np.cov` path) and fits with one estimator (`np.polyfit`; the old `cov(ddof=1)/var(ddof=0)` mix was off by $N/(N-1)$). NaN below 3 aligned pairs.

`stretched_weeks(perf, strategy)` — score dates whose settled row spans more than one rebalance (`exit - entry > STRETCH_DAYS = 9` calendar days): a skipped Friday makes `settle_pending` exit off the 2-weeks-later successor, so ~10 trading days settle as one row. Such rows stay in the record table but are excluded from all weekly inference (CI, MDE, Sharpe, battery, beta) and disclosed with a count; the missing week itself already fails V-PROCESS. Adopted 2026-07-08, 0 settled weeks.

Book analytics

`weekly_table(perf)` — `performance.csv` pivoted to `score_date` rows × strategy columns of net returns. The report's section A.

`benchmark_weekly_returns(bench_closes, perf)` — for each settled week (taken from the SPY rows' entry/exit dates), each benchmark ETF's entry→exit return via `paper._price_asof` — the same price convention as settlement, so the comparison is apples-to-apples.

`sector_concentration(positions, sectors)` — per (week, strategy) book: top sector, its weight share, and the **HHI** (Herfindahl index = $\sum \text{share}^2$ — 1.0 means all one sector, $1/11 \approx 0.09$ means evenly spread). Answers "is the 'diversified' book secretly one big tech bet?"

`_reconstructed_features(closes, date)` — approximate `mom_252/vol_63` z-scores rebuilt from the price cache, for books recorded before the diagnostics archive existed. Marked `approx` wherever used.

`factor_tilts(positions, diag_dir, closes, strategy)` — per book: the weighted-average `mom_252` and `vol_63` z-scores of its holdings (true snapshot features when available, reconstruction otherwise). A tilt of +1.2 momentum says the book is a momentum bet that week. Books with partial feature coverage renormalize over the covered weight (with a warning); zero coverage skips the book.

Drift & process

`importance_stability(diag_dir)` — walk the snapshot dirs in date order; for each, Spearman rank-correlate its feature-importance vector with the previous snapshot's (`rho_prev`) and the first's (`rho_first`), missing keys defaulting to 0.0. Steady ≈ 1.0 = the model keeps believing the same things; a lurch = investigate.

`realized_ic(diag_dir, fallback_scores, closes, perf, strategy)` — per settled week: Spearman correlation between the scores the model published (snapshot if available, else the current score file marked `approx`) and the demeaned entry→exit returns that then actually happened. The honest live IC — tiny positive is the hope; ~ 0 while the book wins means winning for the wrong reasons.

`regime_whipsaw(positions, table, strategy)` — descriptive whipsaw diagnostic (pre-named 2026-07-10, still 0 settled weeks): per-book exposure series for the strategy → count of exposure **flips** between consecutive recorded books, books at **reduced exposure**, and the mean weekly net of **settled risk-off weeks vs SPY** — what de-risking plus the 0%-cash convention actually cost or saved, read as a planned number instead of a post-hoc search. Feeds report section G; no verdict attaches to it.

`process_check(positions, perf, warnings_path=None)` — the machinery audit: build the Friday grid from first to last recorded date, match by **ISO week** (score dates shift on holidays — 2026-07-02 was a Thursday), list missing weeks, count recorded/settled, and count settlement-anomaly rows in `diagnostics/settle-warnings.csv` whose `explained` column the owner has not filled (amended 2026-07-08, replacing the old manual log-grep step). `ok` = no gaps **and** no unexplained anomalies. Feeds V-PROCESS.

Gate, IO, rendering

`check_gate(fixtures, today=None)` — the handcuffs: before `M1_DATE`, no `--fixtures` → `SystemExit`; a `--fixtures` dir **without the** `SYNTHETIC_FIXTURES` **marker** the generator writes → `SystemExit` too, so the flag can't be pointed at an assembled copy of the real ledger as a side door. Peeking at N=5 is noise you will react to, so peeking is an error, not a temptation.

`fetch_benchmark_closes(start)` — RSP/MTUM aren't in the cache (never holdings), so download the battery's closes at report time via yfinance; on failure fall back to Alpaca's data API via `_fetch_alpaca_closes` (pre-registered fallback — no benchmark shopping later; amended 2026-07-15 at 0 settled weeks from the original stooq CSVs, a path discovered dead because stooq blocks this server).

`_fetch_alpaca_closes(tickers, start)` — the amended fallback: one paged request loop against `data.alpaca.markets/v2/stocks/bars` (IEX feed, `adjustment=all` so closes are split+dividend-adjusted like yfinance `auto_adjust`; keys from `~/.config/alpaca/env`; follows `next_page_token` until exhausted) → wide DataFrame of closes from `start`. Data endpoint only, no trading surface.

`load_inputs(fixture)` — one dict of everything `build_report` needs: positions, performance, closes, sectors, diagnostics dir, fallback scores, benchmark closes — from the fixtures dir when given (files mirror the real layout), else from the real ledger/cache/CLI-time download.

`_fmt(x)` / `_tbl(obj, spec)` — percent formatting; fixed-width table rendering via `to_string` (tabulate isn't installed, and no new dependencies is a rule).

`build_report(inputs) -> str` — assembles the markdown: A record table; B headline (mean diff, bootstrap CI, includes/excludes zero, minimum detectable edge, Sharpe + CI); C benchmark battery + V-FACTOR label; D attribution (OLS beta of champion nets on SPY — joint-aligned single-estimator fit, NaN under 3 points — plus sector concentration and factor tilts); E stability & drift; F process check (missing weeks + unexplained settlement anomalies — no manual steps remain); G regime whipsaw (descriptive, added 2026-07-10); pre-committed verdicts.

`main(argv=None)` — argparse → `check_gate` → `load_inputs` → `build_report` → write `research/results/ml-report.md` (or `--output`).

14. `app.py` — the dashboard (Streamlit, tailnet-only)

Not a library — a script Streamlit re-runs top-to-bottom on every page load.

`_stamp()` fingerprints the output files' mtimes and `load_outputs(stamp)` is `@st.cache_data`-cached on that fingerprint, so data reloads exactly when the weekly run rewrites the files. Eight tabs: **Live Run** (next-cron countdown, tail of `weekly.log`), **Today's Signals** (newest ML scores ranked, baseline alongside), **Backtest** (equity curves, metrics, drawdown), **Predictions** (score→outcome scatter, IC history, regime shading), **Insights** (feature importance, ML-vs-baseline rank agreement, and the trees themselves — `render_tree_html` embedded via `components.html`, or a hint to run `train` if the export is absent), **Detail** (per-ticker drill-down), **Paper Record** (ledger positions + settled performance curves), **Docs** (renders `docs/*.md` live from disk). Runs as systemd unit `quant-dashboard`, bound to the Tailscale IP only.

15. `tests/` — how each piece is proven

One file per module, plus fixtures. The style everywhere: **tiny hand-built inputs with hand-computable answers** — e.g. `test_engine` builds four fake tickers and checks kept-vs-filled names; `test_ml` plants a single predictive feature in noise and

asserts the walk-forward finds it (and that no split violates the purge). `test_paper` runs record→settle→re-settle round-trips on synthetic prices and asserts idempotency and exact costs. `test_prices` (the biggest) attacks the merge: NaNs never clobber, new tickers get full history, empty downloads leave the cache alone. `test_trees` fits a 6-tree model on a planted signal and checks the extraction round-trips: every node is a leaf or a split inside `max_depth`, child counts sum to their parent's, the shares normalise to 1.0, one root per tree, and the payload is JSON-serialisable (numpy scalars would break the pipeline write). Its first test pins the private sklearn node-record fields — that is the tripwire for an upgrade.

`m1_fixtures.make_fixtures(root, n_weeks=14, weekly_edge=0.0, snapshot_from=6, seed=0)` builds a complete fake world in a temp dir — positions/performance CSVs in the real schemas (last 2 weeks unsettled, like reality), closes, sectors, benchmark closes, diagnostics dirs from week `snapshot_from` on (testing both snapshot and fallback paths), with a known `weekly_edge` planted so the statistics can be tested against a known truth. This synthetic world is the only data the M1 report saw during development — that's what "blind-built" means concretely.

16. If you were rebuilding this from zero

The dependency order a from-scratch build wants (each step testable before the next): **config → prices (§3) → indicators (§4) → panel + label (§5) → baseline (§6) → metrics (§8) → engine (§7) → ml (§6) → blend → pipeline (§10) → paper (§9) → weekly.sh + status (§11) → dashboard (§14) → diagnostics (§12) → m1_report (§13)**. At every step the discipline is the same three questions: does any number use information from after its date? is there a control group? can a crash or re-run corrupt state? If you can answer no / yes / no at each step, you've rebuilt the system — everything else is pandas.

Pairs with `internals.md` (concepts), `usage.md` (operations), `now.md` (current state).

Regenerate the PDF: `~/doctools/bin/python ~/doctools/md2pdf.py docs/functions.md docs/functions.pdf "Quant Signals – Function Reference"`.